

FrontierAtlas — System Architecture

Scalable, fault-tolerant AI data ingestion, normalization, enrichment, entity resolution, validation, and export pipeline

DATA SOURCES Startups Products arXiv / Papers with Code AI News RSS AI Jobs RSS	ASYNC CRAWLER asyncio + aiohttp Bounded concurrency Timeouts Retries + jitter Per-source controls	RAW / EXTRACTION HTML / RSS / Atom BeautifulSoup + lxml Clean text Metadata extraction	NORMALIZATION URL canonicalization Date normalization Content hashing Freshness filtering Schema validation
	↓	↓	↓
DEDUPLICATION Canonical URL URL hash Content hash First/last seen tracking	ENTITY RESOLUTION Normalization Aliases Exact matching Fuzzy matching Confidence threshold	ENRICHMENT GitHub repository resolution Repository metadata Stars and provenance	LLM ORCHESTRATION Gemini Flash → Groq → DeepSeek Structured JSON 429 backoff 413 chunking Validation
↓	↓	↓	↓
VALIDATED CANONICAL DATA Startups • Products • Research Papers • Jobs • News	STORAGE PostgreSQL target Deterministic IDs Uniqueness constraints Idempotent writes	EXPORT JSON / CSV Excel Google Sheets	FUTURE SCALE LAYER Redis / Kafka queues Horizontal workers Object storage Graph / vector systems

Architecture Principles

Principle	Implementation
Provenance first	Every collected record retains a legitimate source URL; missing optional information is preferred over fabricated data.
Deterministic processing	Parsing, normalization, freshness filtering, hashing, deduplication, and validation are deterministic wherever possible.

Bounded concurrency	Async crawling uses controlled concurrency, timeouts, and source-aware retry behavior.
Fault tolerance	Transient HTTP failures use bounded exponential backoff and jitter; one failing source should not stop other collectors.
Conservative entity resolution	Exact and fuzzy matching are combined with confidence thresholds; low-confidence entities are not blindly merged.
Freshness correctness	published_at and collected_at are kept separate; news and jobs use a strict previous-24-hour freshness rule.
Resumability	Checkpoint files allow long-running collection jobs to resume rather than restarting from scratch.

Collection and Processing Flow

1. Discover	Identify source pages, feeds, APIs, or repositories appropriate for each dataset.
2. Fetch	Use asynchronous HTTP collection with bounded concurrency, timeouts, retryable status handling, and jitter.
3. Parse	Extract HTML/RSS/Atom content and source metadata using deterministic parsers.
4. Clean	Remove boilerplate such as scripts, styles, navigation, advertisements, and irrelevant page elements before semantic processing.
5. Normalize	Canonicalize URLs, normalize dates, names, whitespace, punctuation, and other identifiers.
6. Filter	Apply freshness rules and reject records that fail required validation.
7. Deduplicate	Use canonical URLs, URL hashes, content hashes, and entity identity to prevent duplicate records.
8. Enrich	Resolve legitimate GitHub repositories for research papers and retrieve repository metadata where available.
9. Resolve entities	Map raw names to canonical entities using exact, alias, and conservative fuzzy matching.
10. Validate	Check required fields, provenance, dates, uniqueness, schemas, and enrichment validity.
11. Store / export	Persist canonical records and export them to JSON/CSV/Excel/Google Sheets.

LLM Orchestration

The architecture treats LLM extraction as a controlled enrichment/extraction layer rather than the sole source of truth. A provider fallback chain is designed around Gemini Flash → Groq → DeepSeek. Rate limits such as HTTP 429 trigger bounded exponential backoff with jitter and provider failover. Oversized requests such as HTTP 413 are handled through chunking or reduced payloads rather than blind retries. Structured output is validated before records enter the canonical dataset.

Freshness and Deduplication

Freshness: publication time is extracted with a priority order that favors structured metadata and feed timestamps before visible-page or heuristic signals. **published_at** and **collected_at** remain separate fields. News and jobs are accepted only when published within the previous 24 hours under the project requirement.

Deduplication: URL normalization, URL hashes, content hashes, and entity-level matching reduce duplicate records across sources. The design retains first-seen/last-seen concepts where historical tracking is required.

Fault Tolerance and Scale Path

Current demonstration	Production scaling path	Durable backend	Future analytical layer
Python async collectors Source-specific adapters Checkpointed collection JSON datasets Excel / Sheets export	Scheduler Redis / Kafka queue Crawler workers Extraction workers LLM worker pool Enrichment workers	PostgreSQL Connection pooling Deterministic IDs Uniqueness constraints Idempotent tasks Retry / DLQ	Knowledge graph Vector search Historical changes Observability Incremental updates

Why the Architecture Scales

The collectors, extraction logic, enrichment, entity resolution, and validation are separated into independent stages. This permits worker-level horizontal scaling without rewriting source-specific logic. Idempotent tasks, deterministic identifiers, hashes, uniqueness constraints, bounded retries, and dead-letter queues provide the foundation for processing substantially larger datasets. The target architecture can therefore scale from the demonstration dataset toward hundreds of thousands of records by increasing worker and infrastructure capacity.

Security and Source Access

Credentials are supplied through environment variables and should not be committed to source control. The source-access strategy prefers official APIs, RSS/Atom feeds, sitemaps, and normal HTTP access before browser automation where permitted. The project does not attempt to bypass CAPTCHAs or defeat access-control mechanisms.

Repository Mapping

crawler/ — asynchronous HTTP collection and retry behavior **extraction/** — parsing, freshness, and record handling **entity/** — canonicalization and entity resolution **github/** — GitHub API enrichment **startups/, products/, research/, news/, jobs/** — source-specific pipelines **tests/** — pipeline and component tests